

PYTHON PRACTICE

파이썬 캡슐화 실전: **@property**와 `__`변수

"데이터를 안전하고 우아하게 지키는 법"

Chapter 4. Encapsulation & Property

참고: 타 언어의 캡슐화 (Java)

"보통은 **private** 키워드로 접근을 원천 봉쇄합니다."

Student.java

```
public class Student {  
    // private: 나만 볼 수 있음 (외부 접근 금지)  
    private int gpa;  
  
    // 값을 가져가는 통로 (Getter)  
    public int getGpa() {  
        return gpa;  
    }  
  
    // 값을 수정하는 통로 (Setter)  
    public void setGpa(int value) {  
        this.gpa = value;  
    }  
}
```



엄격한 통제

- `obj.gpa` ➡ **에러!** (접근 불가)
- 반드시 `get/set` 메서드 사용
- 문법적으로 강제됨

"그럼 파이썬은 어떻게 할까요?"

1. 왜 데이터를 숨겨야 할까? (사고 발생)

"변수가 외부에 노출되어 있으면, 누구나 맘대로 바꿀 수 있습니다."



Student (철수)

self.gpa = 4.5



악의적 접근 / 실수

```
# 직접 접근해서 말도 안 되는 값 대입  
chulsu.gpa = 999.0
```

"학점이 999점? 무결성 붕괴!"

2. 파이썬의 해결책 ①: 비공개 변수

"변수 이름 앞에 `__` (언더바 2개)를 붙이면 숨겨집니다."



Private Variable

```
self.__gpa
```

❌ 외부 접근 차단:

`student.__gpa` (에러 발생)

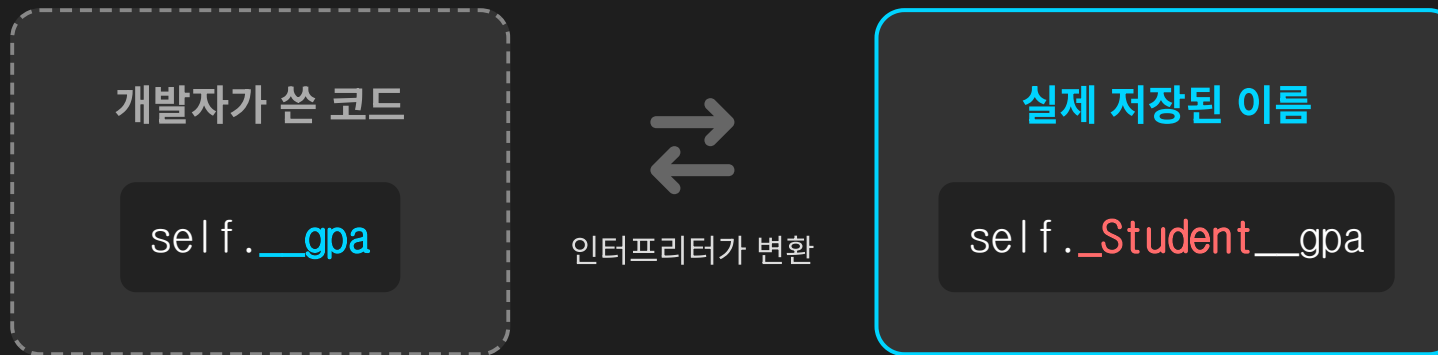
✅ 내부 접근 가능:

클래스 내부 메서드에서는 자유롭게 사용 가능

* 참고: 파이썬은 완벽한 Private은 없습니다. (아래 Name Mangling 참조)

2-1. 비밀의 실체: Name Mangling

"파이썬은 변수를 숨기는 게 아니라, 이름을 바꿔치기 할 뿐입니다."



Name Mangling (네임 맹글링)

외부에서 `_클래스명__` 변수로 접근하면 접근할 수 있습니다.

따라서 완벽한 보안이 아니라, "건드리지 말라"는 강력한 경고입니다.

3. 파이썬의 해결책 ②: @property

"Getter/Setter 메서드를 만들지 마세요. 변수처럼 쓰게 해주세요."

타 언어 방식 (Java Style)

```
class Student:
    def get_gpa(self):
        return self.__gpa

    def set_gpa(self, val):
        self.__gpa = val

# 호출 (불편함)
obj.set_gpa(4.0)
val = obj.get_gpa()
```

"Getter/Setter 정의하느라 코드가 길어짐"

파이썬 방식 (Pythonic)

```
class Student:
    @property
    def gpa(self): return self.__gpa

    @gpa.setter
    def gpa(self, value):
        if value < 0: return # 검사
        self.__gpa = value

# 호출 (편안함)
obj.gpa = 4.0      # 변수처럼 대입
val = obj.gpa     # 변수처럼 조회
```

"변수처럼 쓰지만 실제론 메서드 실행!"